

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao
Princeton University

Dian Yu
Google DeepMind

Jeffrey Zhao
Google DeepMind

Izhak Shafran
Google DeepMind

Thomas L. Griffiths
Princeton University

Yuan Cao
Google DeepMind

Karthik Narasimhan
Princeton University

Abstract

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference. This means they can fall short in tasks that require exploration, strategic lookahead, or where initial decisions play a pivotal role. To surmount these challenges, we introduce a new framework for language model inference, “Tree of Thoughts” (ToT), which generalizes over the popular “Chain of Thought” approach to prompting language models, and enables exploration over coherent units of text (“thoughts”) that serve as intermediate steps toward problem solving. ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices. Our experiments show that ToT significantly enhances language models’ problem-solving abilities on three novel tasks requiring non-trivial planning or search: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%. Code repo with all prompts: <https://github.com/princeton-nlp/tree-of-thought-llm>.

1 Introduction

Originally designed to generate text, scaled-up versions of language models (LMs) such as GPT [25, 26, 1, 23] and PaLM [5] have been shown to be increasingly capable of performing an ever wider range of tasks requiring mathematical, symbolic, commonsense, and knowledge reasoning. It is perhaps surprising that underlying all this progress is still the original autoregressive mechanism for generating text, which makes token-level decisions one by one and in a left-to-right fashion. Is such a simple mechanism sufficient for a LM to be built toward a general problem solver? If not, what problems would challenge the current paradigm, and what should be alternative mechanisms?

The literature on human cognition provides some clues to answer these questions. Research on “dual process” models suggests that people have two modes in which they engage with decisions – a fast, automatic, unconscious mode (“System 1”) and a slow, deliberate, conscious mode (“System 2”) [30, 31, 16, 15]. These two modes have previously been connected to a variety of mathematical models used in machine learning. For example, research on reinforcement learning in humans and other animals has explored the circumstances under which they engage in associative “model free” learning or more deliberative “model based” planning [7]. The simple associative token-level choices of LMs are also reminiscent of “System 1”, and thus might benefit from augmentation by a more deliberate “System 2” planning process that (1) maintains and explores diverse alternatives for current

思维树：利用大型语言模型进行深思熟虑的问题解决

Shunyu Yao 普
林斯顿大学

Dian Yu谷
歌 DeepMind

Jeffrey Zhao谷
歌 DeepMind

Izhak
ShafranGoogle
DeepMind

托马斯·L·格里菲斯
普林斯顿大学

Yuan CaoGoogle
DeepMind

Karthik Narasimhan
普林斯顿大学

摘要

语言模型在广泛任务中的通用问题解决能力日益增强，但在推理过程中仍然局限于逐个标记的左到右决策。这意味着它们在需要探索、战略性前瞻或初始决策起关键作用的任务中可能表现不足。为克服这些挑战，我们提出了一种新的语言模型推理框架——“Thoughts树”（Tree of Thoughts, ToT），该框架在流行的“Chain of Thought”提示方法基础上进行了扩展，支持在连贯的文本单元（“思考”）上进行探索，这些单元作为解决问题的中间步骤。ToT使语言模型能够通过考虑多条不同的推理路径、自我评估选择来进行有意的决策，并在必要时进行前瞻或回溯，以做出全局性决策。我们的实验表明，ToT显著提升了语言模型在三个需要复杂规划或搜索的创新任务中的问题解决能力：24点游戏、创意写作和迷你填字游戏。例如，在24点游戏中，采用链式思考提示的GPT-4仅解决了4%的任务，而我们的方法达到了74%的成功率。所有提示的代码仓库：<https://github.com/princeton-nlp/tree-of-thought-llm>。

1 引言

最初设计用于生成文本的语言模型（LMs）如GPT [25,26, 1, 23]和PaLM [5]的规模扩大后，已被证明在执行越来越广泛的任务方面表现出色，这些任务涉及数学、符号、常识和知识推理。令人或许惊讶的是，所有这些进展的基础仍然是最初的自回归文本生成机制，该机制逐个令牌地进行决策，采用从左到右的方式。这种简单的机制是否足以构建一个面向通用问题解决的语言模型？如果不够，哪些问题会挑战当前的范式，应该采用哪些替代机制？

《思维之树：利用大型语言模型进行深思熟虑的问题解决》关于人类认知的文献为回答这些问题提供了一些线索。关于“双过程”模型的研究表明，人们在决策过程中具有两种模式——一种快速、自动、无意识的模式（“系统1”）和一种缓慢、深思熟虑、意识的模式（“系统2”）[30, 31, 16, 15]。这两种模式此前已被与机器学习中使用的各种数学模型联系起来。例如，关于人类和其他动物的强化学习的研究探讨了它们在何种情况下会进行联想的“无模型”学习或更具深思熟虑的“基于模型”的规划[7]。语言模型中简单的联想符号级选择也让人联想到“系统1”，因此可能通过引入更具深思熟虑的“系统2”规划过程得到增强，该过程（1）维护并探索当前的多样化备选方案。

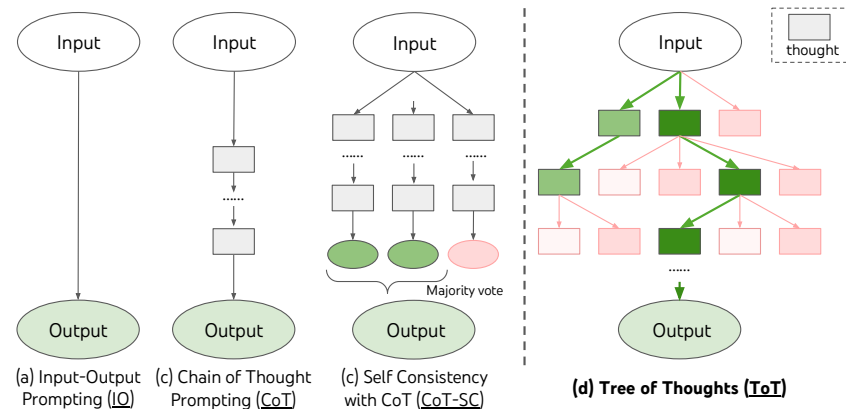


Figure 1: Schematic illustrating various approaches to problem solving with LLMs. Each rectangle box represents a *thought*, which is a coherent language sequence that serves as an intermediate step toward problem solving. See concrete examples of how thoughts are generated, evaluated, and searched in Figures 2,4,6.

choices instead of just picking one, and (2) evaluates its current status and actively looks ahead or backtracks to make more global decisions.

To design such a planning process, we return to the origins of artificial intelligence (and cognitive science), drawing inspiration from the planning processes explored by Newell, Shaw, and Simon starting in the 1950s [21, 22]. Newell and colleagues characterized problem solving [21] as search through a combinatorial problem space, represented as a tree. We thus propose the Tree of Thoughts (ToT) framework for general problem solving with language models. As Figure 1 illustrates, while existing methods (detailed below) sample continuous language sequences for problem solving, ToT actively maintains a tree of thoughts, where each *thought* is a coherent language sequence that serves as an intermediate step toward problem solving (Table 1). Such a high-level semantic unit allows the LM to self-evaluate the progress different intermediate thoughts make towards solving the problem through a deliberate reasoning process that is also instantiated in language (Figures 2,4,6). This implementation of search heuristics via LM self-evaluation and deliberation is novel, as previous search heuristics are either programmed or learned. Finally, we combine this language-based capability to generate and evaluate diverse thoughts with search algorithms, such as breadth-first search (BFS) or depth-first search (DFS), which allow systematic exploration of the tree of thoughts with lookahead and backtracking.

Empirically, we propose three new problems that challenge existing LM inference methods even with the state-of-the-art language model, GPT-4 [23]: Game of 24, Creative Writing, and Crosswords (Table 1). These tasks require deductive, mathematical, commonsense, lexical reasoning abilities, and a way to incorporate systematic planning or search. We show ToT obtains superior results on all three tasks by being general and flexible enough to support different levels of thoughts, different ways to generate and evaluate thoughts, and different search algorithms that adapt to the nature of different problems. We also analyze how such choices affect model performances via systematic ablations and discuss future directions to better train and use LMs.

2 Background

We first formalize some existing methods that use large language models for problem-solving, which our approach is inspired by and later compared with. We use p_θ to denote a pre-trained LM with parameters θ , and **lowercase letters** $x, y, z, s, \dots$ **to denote a language sequence**, i.e. $x = (x[1], \dots, x[n])$ where each $x[i]$ is a token, so that $p_\theta(x) = \prod_{i=1}^n p_\theta(x[i]|x[1..i])$. We use uppercase letters $S, \dots$ to denote a collection of language sequences.

Input-output (IO) prompting is the most common way to turn a problem input x into output y with LM: $y \sim p_\theta(y|\text{prompt}_{IO}(x))$, where $\text{prompt}_{IO}(x)$ wraps input x with task instructions and/or few-shot input-output examples. For simplicity, let us denote $p_\theta^{\text{prompt}}(\text{output} | \text{input}) = p_\theta(\text{output} | \text{prompt}(\text{input}))$, so that IO prompting can be formulated as $y \sim p_\theta^{IO}(y|x)$.

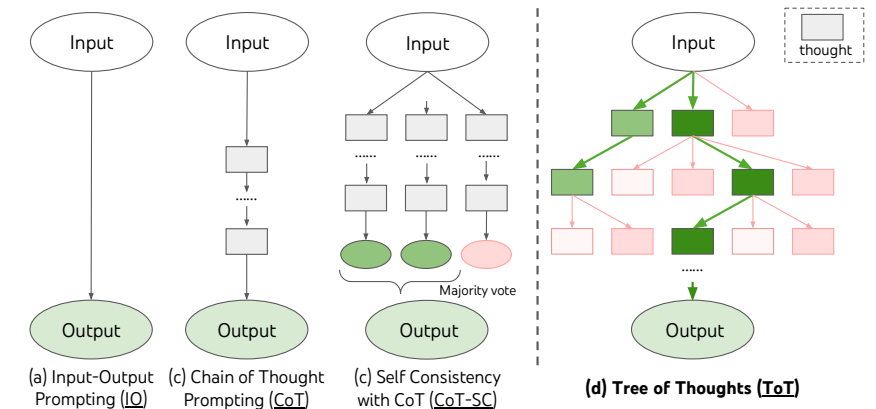


图1: 展示使用大规模语言模型进行问题解决的各种方法的示意图。每个矩形框代表一个思考，即作为问题解决中间步骤的连贯语言序列。有关思考的生成、评估和搜索的具体示例，请参见图2、4、6。

选择而不仅仅是选择一个，(2) 评估其当前状态，并主动向前或回溯以做出更全面的决策。

为了设计这样一个规划过程，我们回溯到人工智能（以及认知科学）的起源，借鉴自1950年代由纽厄尔、肖和西蒙探索的规划过程 [21, 22]。纽厄尔及其同事将问题解决 [21] 描述为在一个组合问题空间中的搜索，该空间以树状结构表示。因此，我们提出了“思维树（Tree of Thoughts, ToT）”框架，用于利用语言模型进行通用问题解决。如图1所示，虽然现有方法（详见下文）通过采样连续的语言序列进行问题解决，ToT则主动维护一棵思维树，其中每个思维都是一个连贯的语言序列，作为解决问题的中间步骤（表1）。这种高层次的语义单元使得语言模型能够通过一种也以语言体现的深思熟虑的推理过程，自我评估不同中间思维在解决问题中的进展（图2、4、6）。通过语言模型的自我评估和深思熟虑实现搜索启发式方法是创新的，因为以往的搜索启发式要么是预编程的，要么是通过学习获得的。最后，我们将这种基于语言的生成和评估多样思维的能力与搜索算法（如广度优先搜索BFS或深度优先搜索DFS）相结合，以系统性地探索思维树，进行前瞻和回溯。

经验上，我们提出了三类新问题，挑战现有的语言模型推理方法，即使是在最先进的语言模型GPT-4 [23]的条件下：24点游戏、创意写作和填字游戏（表1）。这些任务需要演绎推理、数学推理、常识推理、词汇推理能力，以及结合系统性规划或搜索的方法。我们展示了Tree of Thoughts在这三项任务中都取得了优越的结果，原因在于其具有足够的通用性和灵活性，能够支持不同层次的思考、不同的生成与评估思考的方法，以及适应不同问题性质的搜索算法。我们还通过系统性消融分析了这些选择对模型性能的影响，并讨论了未来在更好地训练和应用语言模型方面的研究方向。

2 背景

我们首先形式化一些利用大型语言模型进行问题解决的现有方法，这些方法启发了我们的思路，并在后续进行了比较。我们用 p_θ 表示参数为 θ 的预训练语言模型（LM），用小写字母 $x, y, z, s, \dots$ 表示一段语言序列，即 $x = (x[1], \dots, x[n])$ ，其中每个 $x[i]$ 是一个标记，因此 $p_\theta(x) = \prod_{i=1}^n p_\theta(x[i]|x[1..i])$ 。我们用大写字母 $S, \dots$ 表示一组语言序列。

<Input-output (IO) 提示>是利用大语言模型将问题输入 x 转化为输出 y 的最常用方法： $y \sim p_\theta(y|\text{prompt}_{IO}(x))$ ，其中 $\text{prompt}_{IO}(x)$ 用任务指令和/或少量示例包裹输入 x 。为简便起见，我们用 $p_\theta^{\text{prompt}}(\text{输出} | \text{输入}) = p_\theta(\text{输出} | \text{prompt}(\text{输入}))$ 来表示，因此IO提示可以表述为 $y \sim p_\theta^{IO}(y|x)$ 。

Chain-of-thought (CoT) prompting [38] was proposed to address cases where the mapping of input x to output y is non-trivial (e.g. when x is a math question and y is the final numerical answer). The key idea is to introduce a chain of *thoughts* $z_1, \dots, z_n$ to bridge x and y , where each z_i is a coherent language sequence that serves as a meaningful intermediate step toward problem solving (e.g. z_i could be an intermediate equation for math QA). To solve problems with CoT, each thought $z_i \sim p_{\theta}^{CoT}(z_i \mid x, z_{1\dots i-1})$ is sampled sequentially, then the output $y \sim p_{\theta}^{CoT}(y \mid x, z_{1\dots n})$. In practice, $[z_{1\dots n}, y] \sim p_{\theta}^{CoT}(z_{1\dots n}, y \mid x)$ is sampled as a continuous language sequence, and the **decomposition** of thoughts (e.g. is each z_i a phrase, a sentence, or a paragraph) is left ambiguous.

Self-consistency with CoT (CoT-SC) [36] is an ensemble approach that samples k i.i.d. chains of thought: $[z_{1\dots n}^{(i)}, y^{(i)}] \sim p_{\theta}^{CoT}(z_{1\dots n}, y \mid x)$ ($i = 1 \dots k$), then returns the most frequent output: $\arg \max_y \#\{i \mid y^{(i)} = y\}$. CoT-SC improves upon CoT, because there are generally different thought processes for the same problem (e.g. different ways to prove the same theorem), and the output decision can be more faithful by exploring a richer set of thoughts. However, within each chain there is no local exploration of different thought steps, and the “most frequent” heuristic only applies when the output space is limited (e.g. multi-choice QA).

3 Tree of Thoughts: Deliberate Problem Solving with LM

A genuine problem-solving process involves the repeated use of available information to initiate exploration, which discloses, in turn, more information until a way to attain the solution is finally discovered.—— Newell et al. [21]

Research on human problem-solving suggests that people search through a combinatorial problem-space – a tree where the nodes represent partial solutions, and the branches correspond to operators that modify them [21, 22]. Which branch to take is determined by heuristics that help to navigate the problem-space and guide the problem-solver towards a solution. This perspective highlights two key shortcomings of existing approaches that use LMs to solve general problems: 1) Locally, they do not explore *different* continuations within a thought process – the branches of the tree. 2) Globally, they do not incorporate any type of planning, lookahead, or backtracking to help evaluate these different options – the kind of heuristic-guided search that seems characteristic of human problem-solving.

To address these shortcomings, we introduce *Tree of Thoughts (ToT)*, a paradigm that allows LMs to explore multiple reasoning paths over thoughts (Figure 1(c)). ToT frames any problem as a search over a tree, where each node is a **state** $s = [x, z_{1\dots i}]$ representing a partial solution with the input and the sequence of thoughts so far. A specific instantiation of ToT involves answering four questions: 1. How to **decompose** the intermediate process into thought steps; 2. How to **generate** potential thoughts from each state; 3. How to heuristically **evaluate** states; 4. What **search** algorithm to use.

1. Thought decomposition. While CoT samples thoughts coherently without explicit decomposition, ToT leverages problem properties to design and decompose intermediate thought steps. As Table 1 shows, depending on different problems, a thought could be a couple of words (Crosswords), a line of equation (Game of 24), or a whole paragraph of writing plan (Creative Writing). In general, a thought should be “small” enough so that LMs can generate promising and diverse samples (e.g. generating a whole book is usually too “big” to be coherent), yet “big” enough so that LMs can evaluate its prospect toward problem solving (e.g. generating one token is usually too “small” to evaluate).

2. Thought generator $G(p_{\theta}, s, k)$. Given a tree state $s = [x, z_{1\dots i}]$, we consider two strategies to generate k candidates for the next thought step:

- (a) **Sample** i.i.d. thoughts from a CoT prompt (Creative Writing, Figure 4): $z^{(j)} \sim p_{\theta}^{CoT}(z_{i+1} \mid s) = p_{\theta}^{CoT}(z_{i+1} \mid x, z_{1\dots i})$ ($j = 1 \dots k$). This works better when the thought space is rich (e.g. each thought is a paragraph), and i.i.d. samples lead to diversity;
- (b) **Propose** thoughts sequentially using a “propose prompt” (Game of 24, Figure 2; Crosswords, Figure 6): $[z^{(1)}, \dots, z^{(k)}] \sim p_{\theta}^{propose}(z_{i+1}^{(1\dots k)} \mid s)$. This works better when the thought space is more constrained (e.g. each thought is just a word or a line), so proposing different thoughts in the same context avoids duplication.

3. State evaluator $V(p_{\theta}, S)$. Given a frontier of different states, the state evaluator evaluates the progress they make towards solving the problem, serving as a *heuristic* for the search algorithm to determine which states to keep exploring and in which order. While heuristics are a standard approach to solving search problems, they are typically either programmed (e.g. DeepBlue [3]) or

链式思维（CoT）提示 [38] 被提出以应对输入 x 到输出 y 的映射非平凡的情况（例如当 x 是数学问题而 y 是最终的数值答案时）。其核心思想是引入一系列思维 $z_1, \dots, z_n$ ，以连接 x 和 y ，其中每个 z_i 都是一个连贯的语言序列，作为解决问题的有意义的中间步骤（例如， z_i 可以是数学问答的中间方程）。为了用CoT解决问题，每个思维 $z_i \sim p_{\theta}^{CoT}(z_i \mid x, z_{1\dots i-1})$ 依次采样，然后输出 $y \sim p_{\theta}^{CoT}(y \mid x, z_{1\dots n})$ 。在实际操作中， $[z_{1\dots n}, y] \sim p_{\theta}^{CoT}(z_{1\dots n}, y \mid x)$ 作为连续的语言序列进行采样，而思维的**分解**（例如每个 z_i 是短语、句子还是段落）则保持模糊不清。

带自治性的链式思考（CoT-SC） [36] 是一种集成方法，它采样 k 个相互独立的思考链：

$[z_{1\dots n}^{(i)}, y^{(i)}] \sim p_{\theta}^{CoT}(z_{1\dots n}, y \mid x)$ ($i = 1 \dots k$)，然后返回最频繁的输出： $\arg \max_y \#\{i \mid y^{(i)} = y\}$ 。CoT-SC优于CoT，因为对于同一问题通常存在不同的思考过程（例如证明同一定理的不同方法），通过探索更丰富的思考集合，输出决策可以更具忠实性。然而，在每条链中没有对不同思考步骤的局部探索，“最频繁”启发式仅在输出空间有限时（如多项选择问答）适用。

3 树状思维：利用大型语言模型进行深思熟虑的问题解决

真正的问题解决过程涉及反复利用可用信息进行探索，这反过来会揭示更多信息，直到最终找到实现解决方案的方法。—— Newell 等人 [21]

关于人类问题解决的研究表明，人们在组合问题空间中进行搜索——一个树状结构，其中节点代表部分分解，分支对应修改这些部分分解的操作 [21, 22]。选择哪条分支由启发式方法决定，这些方法有助于导航问题空间并引导问题解决者朝向解决方案。这一观点突出了现有利用大语言模型（LM）解决一般性问题的方法的两个关键不足之处：1) 在局部层面，它们未能探索思维过程中的不同连续性——即树的不同分支；2) 在全局层面，它们未能结合任何形式的规划、前瞻或回溯，以帮助评估这些不同的选项——这类启发式引导的搜索似乎是人类问题解决的典型特征。

为了解决这些不足，我们引入思维树（*Tree of Thoughts, ToT*），这是一种允许大语言模型在思维路径上探索多条推理路径的范式（图1(c)）。ToT将任何问题框架为在一棵树上的搜索，其中每个节点是一个**状态**， $s = [x, z_{1\dots i}]$ 代表具有输入和至今为止的思维序列的部分分解。ToT的具体实现涉及回答四个问题：1. 如何将中间过程分解为思维步骤；2. 如何从每个状态生成潜在的思维；3. 如何启发式地**评估**状态；4. 使用何种**搜索**算法。

1. 思维分解。虽然链式推理（CoT）样例中的思维连贯，但未进行显式的分解，ToT则利用问题的属性设计并分解中间思维步骤。如表1所示，根据不同的问题，一个思维可以是几个词（填字游戏）、一行方程（24点游戏）或一段写作计划（创意写作）。一般而言，一个思维应当“足够小”，以便语言模型（LM）能够生成有潜力且多样的样本（例如，生成整本书通常过于“庞大”而难以连贯），同时又“足够大”，以便LM能够评估其在问题解决中的前景（例如，生成一个标记通常过于“微小”而难以评估）。

2. 思维生成器 $G(p_{\theta}, s, k)$. 给定树状态 $s = [x, z_{1\dots i}]$ ，我们考虑两种策略来生成下一步思考的候选项：

- (a) **Sample** 来自CoT提示的独立同分布思维（创造性写作，图

4 $z^{(j)} \sim \cdot$) : () () ()。当

$p_{\theta}^{CoT} z_{i+1} \mid s = p_{\theta}^{CoT} z_{i+1} \mid x z_{1\dots i} j = 1 \dots k$ 思维空间丰富时效果更佳（例如每个思维为一段），而独立同分布样本可以带来多样性；(b) **提出**通过“提出提示”依次提出思维（24点游戏，图2；填字游戏，图6）： $[z^{(1)}, \dots, z^{(k)}] \sim p_{\theta}^{propose}(z_{i+1}^{(1\dots k)} \mid s)$ 。当思维空间更受限制时效果更佳（例如每个思维仅为一个词或一行），在相同上下文中提出不同的思维可以避免重复。

3. 状态评估器 $V(p_{\theta}, S)$. 在给定不同状态的前沿时，状态评估器评估它们在解决问题方面的进展，作为搜索算法的启发式依据，以决定哪些状态应继续探索以及探索的顺序。虽然启发式方法是解决搜索问题的常用策略，但它们通常是通过编程实现的（例如 DeepBlue [3]）或

learned (e.g. AlphaGo [29]). We propose a third alternative, by using the LM to deliberately reason about states. When applicable, such a deliberate heuristic can be more flexible than programmed rules, and more sample-efficient than learned models. Similar to the thought generator, we consider two strategies to evaluate states either independently or together:

- (a) **Value** each state independently: $V(p_\theta, S)(s) \sim p_\theta^{value}(v|s) \forall s \in S$, where a value prompt reasons about the state s to generate a scalar value v (e.g. 1-10) or a classification (e.g. sure/likely/impossible) that could be heuristically turned into a value. The basis of such evaluative reasoning can vary across problems and thought steps. In this work, we explore evaluation via few *lookahead* simulations (e.g. quickly confirm that 5, 5, 14 can reach 24 via $5 + 5 + 14$, or “hot_1” can mean “inn” via filling “e” in “_”) plus commonsense (e.g. 1 2 3 are too small to reach 24, or no word can start with “tzxc”). While the former might promote “good” states, the latter could help eliminate “bad” states. Such valuations do not need to be perfect, and only need to be approximately helpful for decision making.
- (b) **Vote** across states: $V(p_\theta, S)(s) = \mathbb{1}[s = s^*]$, where a “good” state $s^* \sim p_\theta^{vote}(s^*|S)$ is voted out based on deliberately comparing different states in S in a vote prompt. When problem success is harder to directly value (e.g. passage coherency), it is natural to instead compare different partial solutions and vote for the most promising one. This is similar in spirit to a “step-wise” self-consistency strategy, i.e. cast “which state to explore” as a multi-choice QA, and use LM samples to vote for it.

For both strategies, we could prompt the LM multiple times to aggregate the value or vote results to trade time/resource/cost for more faithful/robust heuristics.

Algorithm 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)	Algorithm 2 ToT-DFS($s, t, p_\theta, G, k, V, T, v_{th}$)
Require: Input x , LM p_θ , thought generator $G()$ & size limit k , states evaluator $V()$, step limit T , breadth limit b . $S_0 \leftarrow \{x\}$ for $t = 1, \dots, T$ do $S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in G(p_\theta, s, k)\}$ $V_t \leftarrow V(p_\theta, S'_t)$ $S_t \leftarrow \arg \max_{S \subset S'_t, S =b} \sum_{s \in S} V_t(s)$ end for return $G(p_\theta, \arg \max_{s \in S_T} V_T(s), 1)$	Require: Current state s , step t , LM p_θ , thought generator $G()$ and size limit k , states evaluator $V()$, step limit T , threshold v_{th} if $t > T$ then record output $G(p_\theta, s, 1)$ end if for $s' \in G(p_\theta, s, k)$ do $\triangleright$ sorted candidates if $V(p_\theta, \{s'\})(s) > v_{thres}$ then $\triangleright$ pruning DFS($s', t + 1$) end if end for

4. Search algorithm. Finally, within the ToT framework, one can plug and play different search algorithms depending on the tree structure. We explore two relatively simple search algorithms and leave more advanced ones (e.g. A* [11], MCTS [2]) for future work:

- (a) **Breadth-first search (BFS)** (Algorithm 1) maintains a set of the b most promising states per step. This is used for Game of 24 and Creative Writing where the tree depth is limit ($T \leq 3$), and initial thought steps can be evaluated and pruned to a small set ($b \leq 5$).
- (b) **Depth-first search (DFS)** (Algorithm 2) explores the most promising state first, until the final output is reached ($t > T$), or the state evaluator deems it impossible to solve the problem from the current s ($V(p_\theta, \{s'\})(s) \leq v_{th}$ for a value threshold v_{th}). In the latter case, the subtree from s is *pruned* to trade exploration for exploitation. In both cases, DFS *backtracks* to the parent state of s to continue exploration.

Conceptually, ToT has several benefits as a method for general problem-solving with LMs: (1) *Generality*. IO, CoT, CoT-SC, and self-refinement can be seen as special cases of ToT (i.e. trees of limited depth and breadth; Figure 1). (2) *Modularity*. The base LM, as well as the thought decomposition, generation, evaluation, and search procedures can all be varied independently. (3) *Adaptability*. Different problem properties, LM capabilities, and resource constraints can be accommodated. (4) *Convenience*. No extra training is needed, just a pre-trained LM is sufficient. The next section will show how these conceptual benefits translate to strong empirical performance in different problems.

4 Experiments

We propose three tasks that are hard even when sampling from the state-of-the-art language model, GPT-4 [23], using standard IO prompting or chain-of-thought (CoT) prompting. We show how

学习 (例如 AlphaGo [29])。我们提出第三种方案，即利用语言模型 (LM) 有意地对状态进行推理。在适用的情况下，这种有意的启发式方法比预设规则更具灵活性，也比学习模型更高效。类似于思考生成器，我们考虑两种策略来评估状态：单独评估或联合评估：

- (a) **值独立评估**每个状态: $V(p_\theta, S)(s) \sim p_\theta^{value}(v|s) \forall s \in S$, 其中值提示对状态 s 进行推理以生成一个标量值 v (例如1-10) 或一个分类 (例如确定/可能/不可能), 可以通过启发式方法转化为数值。这种评估推理的基础可以因问题和思考步骤而异。在本工作中, 我们探索通过少量前瞻模拟 (例如快速确认5、5、14是否可以通过 $5 + 5 + 14$ 达到24, 或 “hot 1” 是否意味着 “inn” 通过在 “ ” 中填入 “e”) 加上常识 (例如1、2、3太小, 无法达到24, 或没有单词可以以 “tzxc” 开头) 进行评估。前者可能促进 “良好” 状态, 后者则有助于排除 “坏” 状态。这些估值不需要完美, 只需在决策中大致有帮助。
- (b) **Vote** 跨州投票: $V(p_\theta, S)(s) = \mathbb{1}[s = s^*]$, 其中 “良好” 州 $s^* \sim p_\theta^{vote}(s^*|S)$ 是基于在 S 的投票提示中有意比较不同州而投出的。当问题的成功更难以直接评估 (例如段落连贯性) 时, 采用比较不同部分解决方案并投票选出最有前景的方案是自然的。这与 “逐步” 自治性策略的思想类似, 即将 “探索哪个状态” 作为多项选择问答, 并利用LM样本进行投票。

对于这两种策略, 我们可以多次提示大型语言模型 (LM) 以汇总值或投票结果, 从而在时间/资源/成本与更可靠/稳健的启发式方法之间进行权衡。

算法 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)	算法 2 ToT-DFS($s, t, p_\theta, G, k, V, T, v_{th}$)
Require: 输入 x , LM p_θ , 思维生成器 $G()$ & size limit k , 状态评估器 $V()$, 步骤限制 T , 宽度限制 b . $S_0 \leftarrow \{x\}$ 对于 $t = 1, \dots, T$ do $G(p_\theta, s, k)$ 树状思维: 利用大型语言模型进行深思熟虑的问题解决 $\arg \max_{S \subset S'_t, S =b} \sum_{s \in S} V_t(s)$ 结束循环 Tree of Thoughts: Deliberate Problem Solving with Large Language Models	Require: 当前状态 s , 步骤 t , 大型语言模型 p_θ , 思考生成器 $G()$ 和大小限制 k , 状态评估器 $V()$, 步数限制 T , 阈值 v_{th} if $t > T$ then 记录输出 $G(p_\theta, s, 1)$ 结束条件 对于 $s' \in G(p_\theta, s, k)$ do 排序候选 如果 $V(p_\theta, \{s'\})(s) > v_{thres}$ 则 $\triangleright$ 剪枝 DFS($s', t + 1$) 结束条件 结束循环

4. 搜索算法。最后, 在Tree of Thoughts (思维树) 框架下, 可以根据树结构插入和使用不同的搜索算法。我们探讨了两种相对简单的搜索算法, 留待未来研究探索更为复杂的算法 (例如A* [11], MCTS [2])。

- (a) **广度优先搜索 (BFS)** (算法1) 在每一步中维护一组 b 最具潜力的状态。这用于 “24点游戏” 和创意写作, 其中树的深度有限 ($T \leq 3$), 初始思考步骤可以被评估并剪枝为一小组 ($b \leq 5$)。
- (b) **深度优先搜索 (DFS)** (算法2) 首先探索最有潜力的状态, 直到达到最终输出 ($t > T$), 或状态评估器认为无法从当前的 s ($V(p_\theta, \{s'\})(s) \leq v_{th}$) 解决问题, 满足值阈值 v_{th} 。在后一种情况下, 从 s 开始的子树被剪枝, 以在探索与利用之间进行权衡。在这两种情况下, DFS回溯到 s 的父状态, 继续探索。

从概念上讲, ToT 作为一种通用问题解决方法具有若干优势: (1)通用性。IO、CoT、CoT-SC 和自我优化可以被视为 ToT 的特殊情况 (即有限深度和宽度的树; 图 1)。(2)模块化。基础语言模型 (LM) 以及思维分解、生成、评估和搜索过程都可以独立变化。(3)适应性。可以适应不同的问题特性、LM 能力和资源限制。(4)便利性。无需额外训练, 只需预训练的语言模型即可。下一节将展示这些概念优势如何转化为在不同问题中的强大实证表现。

4 实验

我们提出了三个任务, 即使在使用最先进的语言模型GPT-4 [23], 进行标准输入输出提示或链式思维 (CoT) 提示采样时也依然困难。我们展示了如何

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	74%
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

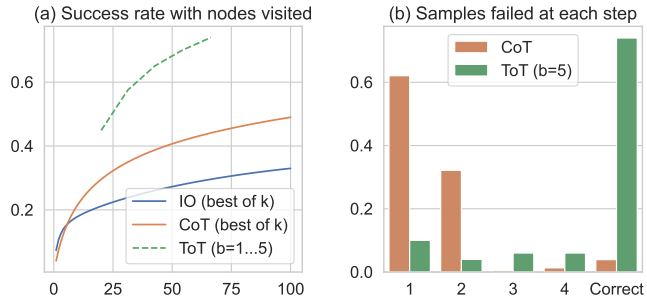


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Results. As shown in Table 2, IO, CoT, and CoT-SC prompting methods perform badly on the task, achieving only 7.3%, 4.0%, and 9.0% success rates. In contrast, ToT with a breadth of $b = 1$ already achieves a success rate of 45%, while $b = 5$ achieves 74%. We also consider an oracle setup for IO/CoT, by calculating the success rate using best of k samples ($1 \leq k \leq 100$). To compare IO/CoT (best of k) with ToT, we consider calculating the tree nodes visited per task in ToT across $b = 1 \dots 5$, and map the 5 success rates in Figure 3(a), treating IO/CoT (best of k) as visiting k nodes in a bandit. Not surprisingly, CoT scales better than IO, and best of 100 CoT samples achieve a success rate of 49%, but still much worse than exploring more nodes in ToT ($b > 1$).

Error analysis. Figure 3(b) breaks down at which step CoT and ToT samples fail the task, i.e. the thought (in CoT) or all b thoughts (in ToT) are invalid or impossible to reach 24. Notably, around 60% of CoT samples already failed the task after generating the first step, or equivalently, the first three words (e.g. “4 + 9”). This highlights the issues with direct left-to-right decoding.

4.2 Creative writing

Next, we invent a creative writing task where the input is 4 random sentences and the output should be a coherent passage with 4 paragraphs that end in the 4 input sentences respectively. Such a task is open-ended and exploratory, and challenges creative thinking as well as high-level planning.

Task setup. We sample random sentences from randomwordgenerator.com to form 100 inputs, and there is no groundtruth passage for each input constraint. As we find that GPT-4 can follow the input constraints most of the time, we focus on evaluating passage coherency in two ways: using a GPT-4 zero-shot prompt to provide a 1-10 scalar score, or using human judgments to compare pairs of outputs from different methods. For the former, we sample 5 scores and average them for each task output, and we find these 5 scores usually consistent, with a standard deviation of around 0.56 on average across outputs. For the latter, we employ a subset of the authors in a blind study to compare the coherency of CoT vs. ToT generated passage pairs, where the order of passages is random flipped over 100 inputs.

Baselines. Given the creative nature of the task, both IO and CoT prompts are zero-shot. While the former prompts the LM to directly generate a coherent passage given input constraints, the latter prompts the LM to first make a brief plan then write the passage, i.e. the plan serves as the intermediate thought step. We generate 10 IO and CoT samples per task. We also consider an iterative-refine ($k \leq 5$) method on top of a random IO sample for each task, where the LM is conditioned on input constraints and the last generated passage to decide if the passage is already “perfectly coherent”, and if not generate a refined one.

ToT setup. We build a ToT with depth 2 (and only 1 intermediate thought step) — the LM first generates $k = 5$ plans and votes for the best one (Figure 4), then similarly generate $k = 5$ passages based on the best plan then vote for the best one. Here the breadth limit $b = 1$, as only one choice is kept per step. A simple zero-shot vote prompt (“analyze choices below, then conclude which is most promising for the instruction”) is used to sample 5 votes at both steps.

Results. Figure 5(a) shows average GPT-4 scores across 100 tasks, where ToT (7.56) is deemed to generate more coherent passages than IO (6.19) and CoT (6.93) on average. While such an automatic metric might be noisy, Figure 5(b) confirms the finding by showing that humans prefer ToT over CoT in 41 out of 100 passage pairs, while only prefer CoT over ToT in 21 (other 38 pairs are found “similarly coherent”). Lastly, iterative-refine is more effective on this natural language task, where

方法	成功
请提供需要翻译的文本内容。	7.3%
CoT提示	4.0%
CoT-SC (k=100)	9.0%
ToT (我们的) (b=1)	45%
Tree of Thoughts: Deliberate Problem Solving with Large Language Models	74%
IO +Refine (k=10)	27%
IO (100分制最佳)	33%
CoT (best of 100)	49%

表2：24点游戏结果。

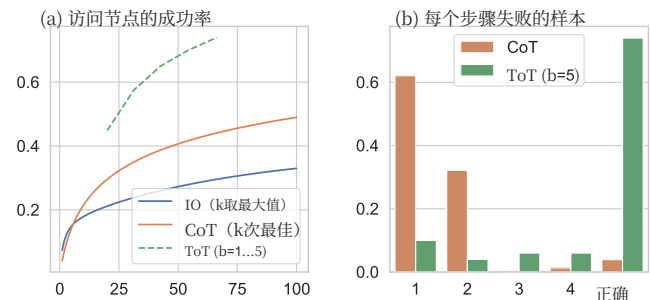


图3：24点游戏 (a) 规模分析与 (b) 误差分析

结果。如表2所示，IO、CoT和CoT-SC提示方法在该任务中的表现较差，成功率分别仅为7.3%、4.0%和9.0%。相比之下，具有 $b = 1$ 宽度的ToT已实现45%的成功率，而 $b = 5$ 则达到了74%。我们还考虑了IO/CoT的oracle设置，通过计算使用 k 个样本中的最佳样本 ($1 \leq k \leq 100$) 的成功率。为了将IO/CoT (k次最佳) 与ToT进行比较，我们考虑在 $b = 1 \dots 5$ 范围内计算ToT中每个任务访问的树节点数，并将图3(a)中的5个成功率映射为在bandit中访问 k 个节点的IO/CoT (k 的最佳) 情况。不出所料，CoT的扩展效果优于IO，100个CoT样本的最佳结果实现了49%的成功率，但仍远远不及在ToT中探索更多节点的效果 ($b > 1$)。

错误分析。 图3(b) 分解了在任务中CoT和ToT样本在哪个步骤失败，即思考（在CoT中）或所有 b 思考（在ToT中）无效或无法达到24。值得注意的是，约60%的CoT样本在生成第一步后就已失败，或等同于生成前三个词（例如“4 + 9”）。这突显了直接从左到右解码存在的问题。

4.2 创意写作

接下来，我们设计一个创意写作任务，输入为4个随机句子，输出应为一篇连贯的段落，每段以这4个句子中的一句结尾。这类任务具有开放性和探索性，既考验创造性思维，也需要高层次的规划能力。

Task setup. 我们从randomwordgenerator.com随机抽取句子，形成100个输入，且每个输入的约束没有对应的标准答案。由于我们发现GPT-4在大多数情况下能够遵循输入约束，因此我们主要通过两种方式评估段落连贯性：一是使用GPT-4的零-shot提示为每个输出提供1到10的标量评分，二是通过人工判断比较不同方法生成的段落对的连贯性。对于前者，我们抽取5个评分并取平均值，发现这5个评分通常具有一致性，平均标准差约为0.56。对于后者，我们采用作者中的一部分人在盲测中比较CoT与ToT生成的段落对的连贯性，段落的顺序在100个输入中随机翻转。

基线。 由于任务的创造性特性，IO和CoT提示均为零样本。前者提示语言模型在给定输入约束的情况下直接生成连贯的段落，后者则提示模型先制定简要计划，然后撰写段落，即计划作为中间思考步骤。我们为每个任务生成10个IO和CoT样本。我们还考虑在每个任务的随机IO样本基础上应用迭代优化 ($k \leq 5$) 方法，其中语言模型在输入约束和最后生成的段落的条件下判断该段落是否已“完美连贯”，如果不是，则生成一个改进版本。

ToT setup. 我们构建了一个深度为2的ToT（仅有1个中间思考步骤）——语言模型首先生成 $k = 5$ 计划并投票选出最佳方案（图4），然后基于最佳方案类似地生成 $k = 5$ 段落，并再次投票选出最佳方案。这里的宽度限制 $b = 1$ ，每个步骤只保留一个选择。采用简单的零-shot投票提示（“分析以下选项，然后得出最符合指令的结论”）在两个步骤中采样5个投票。

结果。Results. 图5(a)显示了100个任务中GPT-4的平均得分，其中ToT (7.56) 被认为比IO (6.19) 和CoT (6.93) 在生成连贯段落方面表现更优。虽然这种自动指标可能存在噪声，但图5(b)通过显示在人类评审中，100对段落中有41对人类更偏好ToT而非CoT，而只有21对偏好CoT而非ToT（其余38对被认为“同样连贯”）来确认这一发现。最后，迭代式细化在此自然语言任务中表现出更高的有效性，

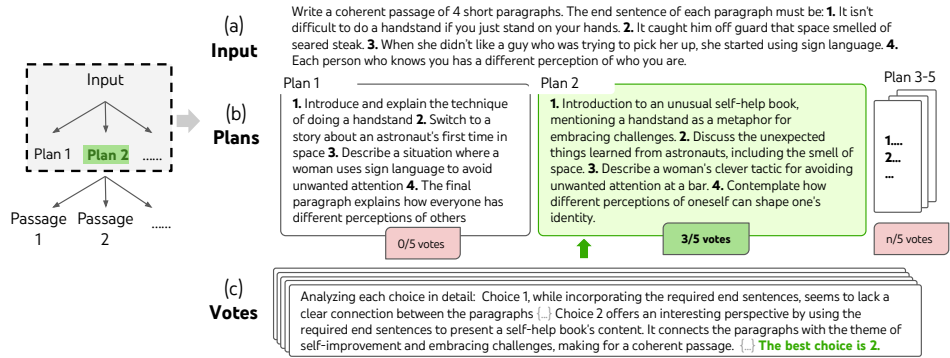


Figure 4: A step of deliberate search in a randomly picked Creative Writing task. Given the input, the LM samples 5 different plans, then votes 5 times to decide which plan is best. The majority choice is used to consequently write the output passage with the same sample-vote procedure.

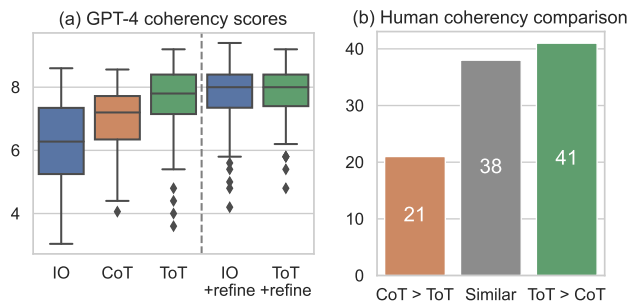


Figure 5: Creative Writing results.

it improves IO coherency score from 6.19 to 7.67, and ToT coherency score from 7.56 to 7.91. We believe it could be thought of as a third approach to thought generation in the ToT framework, where new thoughts can arise from refining old thoughts instead of i.i.d. or sequentially generated.

4.3 Mini crosswords

In Game of 24 and Creative Writing, ToT is relatively shallow — at most 3 thought steps are needed to reach the final output. Here we explore 5×5 mini crosswords as a harder search problem involving natural language. Again, the goal is not just to solve the task, as more general crosswords can be readily solved with specialized NLP pipelines [34] that leverages large-scale retrieval instead of LM. Rather, we aim to explore the limit of LM as a general problem solver that explores its own thoughts and guides its own exploration with deliberate reasoning as heuristics.

Task setup. We scrape data from GooBix, which contains 156 games of 5×5 mini crosswords. As we observe adjacent games contain similar clues, we use 20 games with indices 1, 6, ..., 91, 96 for testing, and games 136, 141, 146, 151, 156 for prompting. For each task, the input describes the 5 horizontal clues and 5 vertical clues, and the output should be a board of $5 \times 5 = 25$ letters to solve the crosswords. For evaluation, we consider three levels of success: the portion of correct letters (25 per game), words (10 per game), and games.

Baselines. We provide 5 example input-output pairs in the IO prompt, and in the CoT prompt additionally add intermediate words in the order h1..5 then v1..5. We run each prompt for 10 samples and average the results.

ToT setup. We leverage a depth-first search (Algorithm 2) that keeps exploring the most promising subsequent word clue until the state is no longer promising, then backtrack to the parent state to explore alternative thoughts. To make search tractable, subsequent thoughts are constrained not to change any filled words or letters, so that the ToT has at most 10 intermediate steps. For thought generation, at each state we translate all existing thoughts (e.g. “h2.motor; h1.tasks” for the state in Figure 6(a)) into letter constraints for remaining clues (e.g. “v1.To heap: tm_...;...””) and prompt a proposal prompt 5 times to come up with candidates for where and what to fill in the next word. Importantly, we also prompt the LM to give a confidence level for different thoughts, and aggregate

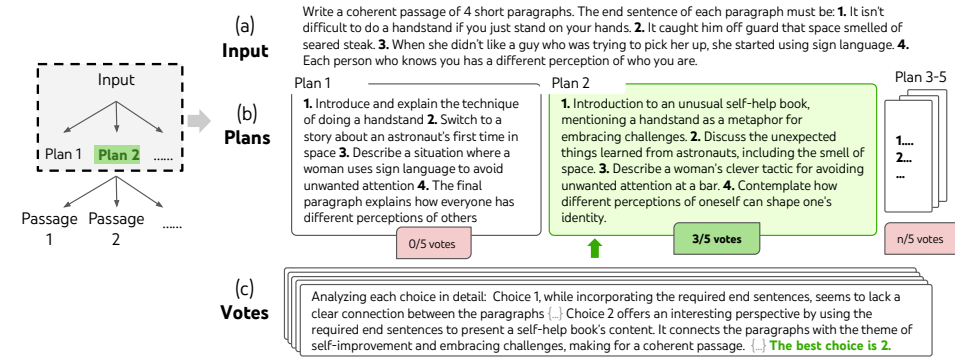


图4：在随机选择的创意写作任务中，有意搜索的一个步骤。给定输入后，语言模型采样5个不同的方案，然后进行5次投票以决定哪个方案最佳。多数选择随后用于生成输出段落，采用相同的采样-投票流程。

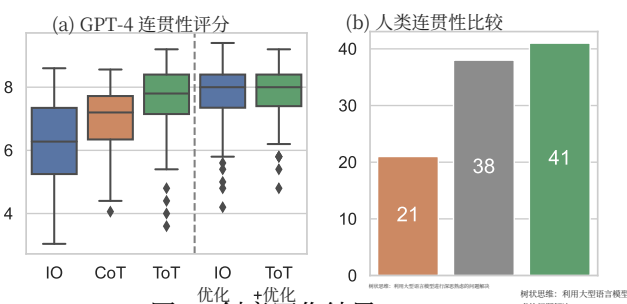


图5：创意写作结果。

它将IO连贯性得分从6.19提升至7.67，ToT连贯性得分从7.56提升至7.91。我们认为这可以被视为ToT框架中思维生成的第三种方法，即新思维可以通过细化旧思维而非独立同分布或顺序生成而产生。

4.3 小型填字游戏

在《24点游戏》和创意写作中，Tree of Thoughts (ToT) 相对浅显——最多只需三步思考即可达到最终输出。在这里，我们将 5×5 迷你填字游戏作为一个涉及自然语言的更难搜索问题进行探讨。同样，目标不仅仅是解决任务，因为更通用的填字游戏可以通过专门的自然语言处理管道 [34] 轻松解决，该管道利用大规模检索而非语言模型 (LM)。而是，我们旨在探索语言模型作为通用问题解决者的极限，它通过有意的推理作为启发式，探索自身的思考并引导自身的探索。

任务设置。我们从GooBix抓取数据，包含 5×5 个迷你填字游戏。由于我们观察到相邻的游戏包含相似的线索，因此使用索引为1、6、91、96的20个游戏进行测试，索引为136、141、146、151、156的游戏用于提示。对于每个任务，输入描述5个横向线索和5个纵向线索，输出应为一个 $5 \times 5 = 25$ 字母的棋盘，用于解答填字游戏。评估时，我们考虑三个成功等级：正确字母的比例（每个游戏25个）、正确单词的比例（每个游戏10个）以及游戏的整体成功率。

基线。我们在 IO 提示中提供了 5 个示例输入输出对，在 CoT 提示中还包括了中间词，顺序为 h1..5 然后 v1..5。我们对每个提示运行 10 个样本并取平均结果。

Tree of Thoughts: Deliberate Problem Solving with Large Language Models
ToT setup. 我们采用深度优先搜索（算法2），不断探索最有潜力的后续词线索，直到该状态不再具有潜力，然后回溯到父状态以探索其他思路。为了使搜索变得可行，后续思路被限制为不改变已填入的单词或字母，从而使Tree of Thoughts最多包含10个中间步骤。在思路生成方面，在每个状态下，我们将所有现有的思路（例如图6(a)中的“h2.motor; h1.tasks”）转化为剩余线索的字母约束（例如“v1.To heap: tm_...;...”），并进行5次提议，生成候选的下一步填入位置和内容。重要的是，我们还会让语言模型给出不同思路的置信度，并进行汇总。

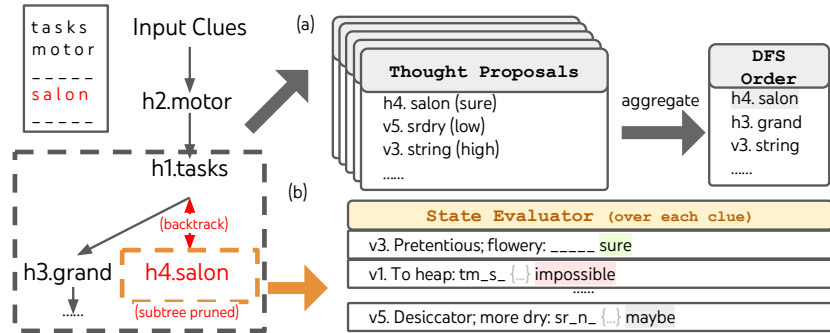


Figure 6: In Mini Crosswords, (a) how thoughts are proposed and aggregated in a priority queue for depth-first search (DFS), and (b) how a state is evaluated based on the possibility of filling in each remaining word clue, and pruned if any remaining clue is deemed not possible to fill by the LM. Then DFS backtracks to the parent state and explore the next promising thought for clue.

these across proposals to obtain a sorted list of next thoughts to explore (Figure 6(a)). For state evaluations, we similarly translate each state into letter constraints for remaining clues, then evaluate for each clue if it is possible to fill given the constraints. If any remaining clue is deemed “impossible” to fill in (e.g. “v1. To heap: tm_s_”), then the exploration of the state’s subtree is pruned and DFS backtracks to its parent to explore the next promising thought. We limit DFS search steps to 100, and simply render the deepest explored state (the first explored one if multiple) into the final output.

Results. As shown in Table 3, IO and CoT prompting methods perform poorly with a word-level success rate less than 16%, while ToT significantly improves all metrics, achieving a word-level success rate of 60% and solving 4 out of 20 games. Such an improvement is not surprising, given IO and CoT lack mechanisms to try different clues, make changes to decisions, or backtrack.

Oracle and ablation studies. When outputting from the oracle best DFS state (instead of the heuristically determined best state) per task, ToT performance is even higher and actually solves 7/20 games (Table 3, “+best state”), indicating our simple output heuristics can be readily improved. Interestingly, sometimes when the crosswords game is actually solved, the state evaluator might still deem some words as “impossible” and prune — possibly because 5×5 crosswords by design have some rare or obsolete words that GPT-4 cannot recognize². Given the state evaluation as a pruning heuristic is imperfect, we also explore ablating the pruning, and find the performance generally worse (Table 3, “-prune”). However, it could actually find the correct solution for 4/20 games (though only outputting 1 via heuristic), 3 of which are games ToT+pruning cannot solve within 100 steps. Thus, better heuristics for DFS pruning are critical for problem solving in this case. Lastly, we confirm the importance of backtracking by running an ablation that keeps filling the most promising clue for at most 20 steps, allowing overwrites. This is similar to a “greedy” BFS search with breadth limit of $b = 1$, and performs poorly with a word level success of only 20% (Table 3, “-backtrack”).

5 Related Work

Planning and decision making. Smart planning and decision making are critical to achieving predefined goals. As they are trained on vast amount of world knowledge and human examples, LMs are known to have already absorbed rich commonsense that makes it possible to propose reasonable plans conditioned on problem setting and environmental states [12, 42, 37, 13, 35, 41, 40]. Our proposed ToT approach extends existing planning formulations by considering multiple potentially feasible plans simultaneously at each problem-solving step, and proceeding with the most promising ones. The integration between thought sampling and value feedback organically integrates planning and decision-making mechanisms, enabling effective search inside a solution tree. On the other hand, traditional decision-making procedures usually require training dedicated reward and policy models as in reinforcement learning (for example CHAI [33]), whereas we use the LM itself to provide the value estimates for decision making. RAP [9] is a concurrent work that treats language model

²For example, “agend” is an obsolete form of “agendum”, but GPT-4 deems it a typo for “agenda”. External retrieval or web interaction could augment LM for problem solving under knowledge uncertainty.

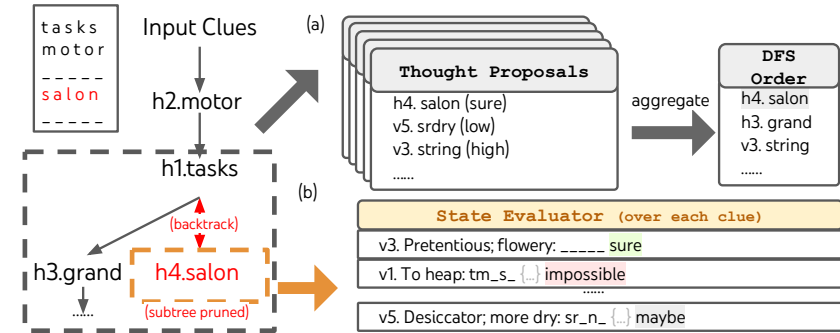


图6：在迷你填字游戏中，（a）思维如何在优先队列中被提出和聚合以进行深度优先搜索（DFS），以及（b）如何根据填充每个剩余线索的可能性对状态进行评估，并在任何剩余线索被判定为无法由语言模型填充时进行剪枝。然后，DFS回溯到父状态并探索下一个有潜力的线索思路。

通过这些提议获得下一步要探索的思考排序列表（图6(a)）。对于状态评估，我们同样将每个状态转换为剩余线索的字母约束，然后评估每个线索在给定约束条件下是否可能填充。如果任何剩余线索被判定为“无法”填充（例如“v1. 堆：tm s_”），则剪枝该状态的子树，回溯到其父节点以探索下一个有希望的思考。我们将深度优先搜索（DFS）的搜索步数限制为100，并将最深探索到的状态（如果有多个，则为第一个）作为最终输出。

结果。 如表3所示，IO和CoT提示方法的表现较差，词级成功率均低于16%，而ToT显著提升了所有指标，达到60%的词级成功率，并解决了20局中的4局。考虑到IO和CoT缺乏尝试不同线索、调整决策或回溯的机制，这样的提升并不令人意外。

Oracle与消融研究。 当从oracle的最佳DFS状态（而非启发式确定的最佳状态）输出每个任务时，ToT的性能甚至更高，实际上解决了7/20个游戏（表3，“+best state”），表明我们的简单输出启发式可以被轻松改进。有趣的是，有时当字谜游戏实际上被解决时，状态评估器仍可能将某些单词判定为“不可行”并进行剪枝——可能是因为 5×5 字谜设计中包含一些GPT-4无法识别的罕见或过时的单词²。鉴于状态评估作为剪枝启发式是不完善的，我们也探索了去除剪枝的效果，发现性能通常较差（表3，“-prune”）。然而，实际上它可以在20个游戏中找到正确的解答4个（尽管仅通过启发式输出1个），其中3个是ToT+在100步内无法解决的游戏。因此，更优的DFS剪枝启发式对于此类问题的解决至关重要。最后，我们通过进行消融实验确认了回溯的重要性，该实验在最多20步内持续填充最有潜力的线索，允许覆盖。这类似于“贪婪”式的宽度优先搜索，宽度限制为 $b = 1$ ，其表现较差，单词层级成功率仅为20%（表3，“-backtrack”）。

5 相关工作

规划与决策制定。 智能规划与决策制定对于实现预设目标至关重要。由于它们在大量世界知识和人类示例的基础上进行训练，语言模型（LMs）已被证明已吸收丰富的常识，从而能够根据问题设定和环境状态 [12, 42, 37, 13, 35, 41, 40]提出合理的方案。我们提出的树的思考（ToT）方法通过在每个问题解决步骤中同时考虑多个潜在可行方案，并优先选择最具潜力的方案，扩展了现有的规划公式。思考采样与价值反馈的有机结合，将规划与决策机制紧密结合，促进在解题树中的高效搜索。另一方面，传统的决策制定程序通常需要训练专门的奖励模型和策略模型，如强化学习中的方法（例如CHAI [33]），而我们则利用语言模型本身提供决策的价值估计。RAP [9] 是一个同时进行的工作，旨在将语言模型

例如，“agend”是“agendum”的过时形式，但GPT-4将其视为“agenda”的拼写错误。外部检索或网络交互可以增强语言模型在知识不确定性下的问题解决能力。

reasoning as planning with its internal world model, and proposes a MCTS-based method similar to ToT. However, its tasks are simpler than ours, and its framework lacks the modularity to incorporate different tree search algorithms.

Self-reflection. Using LLMs to assess the viability of their own predictions is becoming an increasingly important procedure in problem solving. [28, 20, 24] introduced the “self-reflection” mechanism, in which LMs provide feedback to their generation candidates. [4] improves LMs code generation accuracy by injecting feedback messages generated by the LM itself based on its code execution results. Similarly, [17] also introduces “critic” or review steps over the actions and states, deciding the next action to take in solving computer operation tasks. Another recent work very relevant to ours is “self-eval guided decoding” [39]. Similar to our method, self-eval decoding also follows a tree-search procedure with leaves sampled from stochastic beam search decoding, which are then evaluated by LLM itself with carefully prepared self-eval prompts. Their approach however, uses the PAL formulation [8] which represents thoughts as codes, which makes it difficult to tackle challenging tasks like creative writing which we consider in this paper. Our Tree-of-Thought formulation is thus more versatile and handles challenging tasks on which GPT-4 only achieves very low accuracy with standard prompts.

Program-guided LLM generation. Our proposal is also related to recent advancements that organize LM’s behavior with systematic procedures [14, 44, 6, 43] or symbolic program guidance. For example, Schlag et al. [27] embeds LMs in an algorithmic search procedure to help solve problems like question answering step-by-step, in which the search trees are expanded by relevant paragraphs that might provide answers. This approach however differs from ours in that trees are expanded by sampling external paragraphs instead of the LM’s own thoughts, and there is no reflection or voting steps. Another approach, LLM+P [18], goes one step further and delegates the actual planning process to a classical planner.

Classical search methods. Last but not least, our approach can be treated as a modern rendition of classical search methods for problem solving. For example it can be considered as a heuristic search algorithm like A* [10], in which the heuristic at each search node is provided by the LM’s self-assessment. From this perspective, our method is also related to NeuroLogic A*-esque decoding [19], which is inspired by A* search but introduces look-ahead heuristics that are efficient for LMs to improve the beam-search or top-k sampling decoding. This method however is constrained to sentence generation tasks, whereas our framework are designed for complex, multi-step problem solving guarded by value feedback.

6 Discussion

Limitations and future directions. Deliberate search such as ToT might not be necessary for many existing tasks that GPT-4 already excels at (see Appendix B.1), and as an initial step this work only explores three relatively simple tasks that challenges GPT-4 (see Appendix B.2 for some GPT-3.5 experiment results) and calls of better search and planning abilities incorporated with LMs. However, as we begin to deploy LMs for more real-world decision making applications (e.g. coding, data analysis, robotics, etc.), more complex tasks could emerge and present new opportunities to study these research questions. Also, search methods like ToT requires more resources (e.g. GPT-4 API cost) than sampling methods in order to improve task performances, but the modular flexibility of ToT allows users to customize such performance-cost tradeoffs, and ongoing open-source efforts [32] should readily reduce such costs in the near future. More details about cost and efficiency are in Appendix B.3. Lastly, this work focuses on using an off-the-shelf LM, and fine-tuning LMs using a ToT-style high-level counterfactual decision making (e.g. deliberating over potential choices for the next paragraph, instead of predicting the next token) might present opportunities to enhance the problem-solving capabilities of LMs.

Conclusion. The associative “System 1” of LMs can be beneficially augmented by a “System 2” based on searching a tree of possible paths to the solution to a problem. The Tree of Thoughts framework provides a way to translate classical insights about problem-solving into actionable methods for contemporary LMs. At the same time, LMs address a weakness of these classical methods, providing a way to solve complex problems that are not easily formalized, such as creative writing. We see this intersection of LMs with classical approaches to AI as an exciting direction.

将推理视为具有其内部世界模型的规划，并提出了一种类似于ToT的基于蒙特卡洛树搜索（MCTS）的方法。然而，其任务比我们的更为简单，且其框架缺乏引入不同树搜索算法的模块化能力。

自我反思。 使用大型语言模型（LLMs）评估其自身预测的可行性，正成为问题解决中越来越重要的程序。 [28, 20, 24] 引入了“自我反思”机制，其中LLMs为其生成候选提供反馈。 [4] 通过注入由LLM自身基于其代码执行结果生成的反馈信息，提升了LLMs的代码生成准确性。类似地， [17] 也引入了“批评”或审查步骤，针对行动和状态，决定在解决计算机操作任务时采取的下一步行动。另一个与我们工作非常相关的最新研究是“自我评估引导解码” [39]。与我们的方法类似，自我评估解码也遵循树搜索程序，叶节点通过随机束搜索解码采样，然后由LLM自身使用精心准备的自我评估提示进行评估。然而，他们的方法使用了PAL表述 [8]，将思考表示为代码，这使得处理像创造性写作这样具有挑战性的任务变得困难，而我们在本文中考虑的任务。我们的Tree-of-Thoughts（思维树）表述因此更具通用性，能够应对GPT-4在标准提示下仅能达到极低准确率的挑战性任务。

程序引导的大规模语言模型生成。我们的提议也与近期的进展相关，这些进展通过系统性程序 [14, 44, 6, 43] 或符号程序引导来组织语言模型的行为。例如，Schlag 等人 [27] 将语言模型嵌入到一种算法搜索过程中，以帮助逐步解决问题，如问答任务，其中搜索树通过可能提供答案的相关段落进行扩展。然而，这种方法与我们的不同之处在于，树的扩展是通过采样外部段落而非语言模型自身的思考，并且没有反思或投票步骤。另一种方法，LLM+P [18]，更进一步，将实际的规划过程委托给经典规划器。

经典搜索方法。最后但同样重要的是，我们的方法可以被视为经典搜索方法在问题解决中的现代演绎。例如，它可以被看作是一种启发式搜索算法，如A* [10]，其中每个搜索节点的启发式由语言模型的自我评估提供。从这个角度来看，我们的方法也与NeuroLogic A*类解码 [19]相关，该方法受到A*搜索的启发，但引入了前瞻性启发式，旨在提高语言模型在束搜索或Top-k采样解码中的效率。然而，该方法仅限于句子生成任务，而我们的框架则设计用于复杂的多步骤问题解决，受价值反馈的引导。

6 讨论

限制与未来方向。**Limitations and future directions.** 类似ToT的有意搜索对于许多GPT-4已经表现出色的现有任务可能并非必要（见附录B.1），作为初步工作，本研究仅探索了三项相对简单、挑战GPT-4的任务（见附录B.2中的一些GPT-3.5实验结果），并呼吁结合更强搜索与规划能力的语言模型。然而，随着我们开始将语言模型应用于更多实际决策场景（如编码、数据分析、机器人等），可能会出现更复杂的任务，为研究这些问题提供新的机遇。此外，像ToT这样的搜索方法在提升任务性能方面比采样方法需要更多资源（例如GPT-4 API成本），但ToT的模块化灵活性允许用户自定义性能与成本的权衡，持续的开源努力 [32]预计将在不久的将来显著降低相关成本。关于成本与效率的更多细节见附录B.3。最后，本研究专注于使用现成的语言模型，利用类似ToT的高层次反事实决策（例如，权衡下一段的潜在选择，而非预测下一个词）对模型进行微调，可能为提升语言模型的解决问题能力提供新的机会。

结论。**结论。** 语言模型的联想“系统1”可以通过基于搜索可能路径树的“系统2”得到有益的增强。Thoughts树框架提供了一种将经典问题解决洞见转化为当代语言模型可操作方法的途径。同时，语言模型也弥补了这些经典方法的不足，为解决难以形式化的复杂问题提供了可能，例如创造性写作。我们认为，语言模型与经典人工智能方法的结合是一个令人振奋的研究方向。

Broader Impact

ToT is a framework that empowers LMs to more autonomously and intelligently make decisions and solve problems. While current tasks are limited to reasoning and search problems, future applications involving interaction with external environments or humans could bring potential danger, e.g. facilitating harmful uses of LMs. On the other hand, ToT also improves the interpretability of model decisions and the opportunity for human alignment, as the resulting representations are readable, high-level language reasoning instead of implicit, low-level token values.

Acknowledgements

SY and KN acknowledge support from an Oracle Collaborative Research award and the National Science Foundation under Grant No. 2239363. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation. SY is also supported by the Harold W. Dodds Fellowship from Princeton.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[2] C. Browne, E. J. Powley, D. Whitehouse, S. M. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. P. Liebana, S. Samothrakis, and S. Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4:1–43, 2012.

[3] M. Campbell, A. J. Hoane Jr, and F.-h. Hsu. Deep blue. *Artificial intelligence*, 134(1-2):57–83, 2002.

[4] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug, 2023.

[5] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

[6] A. Creswell and M. Shanahan. Faithful reasoning using large language models. *arXiv preprint arXiv:2208.14271*, 2022.

[7] N. D. Daw, Y. Niv, and P. Dayan. Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. *Nature neuroscience*, 8(12):1704–1711, 2005.

[8] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. Pal: Program-aided language models, 2023.

[9] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.

[10] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

[11] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[12] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents, 2022.

[13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.

更广泛的影响

Tree of Thoughts: 利用大型语言模型进行深思熟虑的问题解决框架ToT 是一种赋能大型语言模型（LMs）更自主、更智能地做出决策和解决问题的框架。虽然目前的任务局限于推理和搜索问题，但未来涉及与外部环境或人类交互的应用可能带来潜在风险，例如促进有害用途的LMs。一方面，ToT 还提高了模型决策的可解释性和人类对齐的可能性，因为其生成的表示是可读的、高层次的语言推理，而非隐含的低层次标记值。

致谢

SY 和 KN 感谢获得 Oracle 协作研究奖和国家科学基金会（Grant No. 2239363）的支持。本文中表达的任何观点、研究结果、结论或建议均为作者本人观点，不一定代表国家科学基金会的立场。SY 还获得普林斯顿大学 Harold W. Dodds 奖学金的支持。

参考文献

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell 等。语言模型是少-shot 学习者。神经信息处理系统进展, 33:1877–1901, 2020。[2] C. Browne, E. J. Powley, D. Whitehouse, S. M. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. P. Liebana, S. Samothrakis, 和 S. Colton。蒙特卡洛树搜索方法综述。 *IEEE 计算智能与游戏中的的人工智能杂志*, 4:1–43, 2012。[3] M. Campbell, A. J. Hoane Jr, 和 F.-h. Hsu。深蓝。人工智能, 134(1-2):57–83, 2002。[4] X. Chen, M. Lin, N. Schärli, 和 D. Zhou。教导大型语言模型自我调试, 2023。[5] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann 等。Palm: 利用路径扩展语言模型规模。 *arXiv 预印本 arXiv:2204.02311*, 2022。[6] A. Creswell 和 M. Shanahan。利用大型语言模型实现忠实推理。 *arXiv 预印本 arXiv:2208.14271*, 2022。[7] N. D. Daw, Y. Niv, 和 P. Dayan。前额叶系统与背外侧纹状体系统在行为控制中的基于不确定性的竞争。自然神经科学, 8(12):1704–1711, 2005。[8] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, 和 G. Neubig。PAL: 程序辅助的语言模型, 2023。[9] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, 和 Z. Hu。利用语言模型进行推理即为与世界模型的规划。 *arXiv 预印本 arXiv:2305.14992*, 2023。[10] P. E. Hart, N. J. Nilsson, 和 B. Raphael。启发式确定最小成本路径的形式基础。 *IEEE 系统科学与控制论杂志*, 4(2):100–107, 1968。doi: 10.1109/TSSC.1968.300136。[11] P. E. Hart, N. J. Nilsson, 和 B. Raphael。启发式确定最小成本路径的形式基础。 *IEEE 系统科学与控制论杂志*, 4(2):100–107, 1968。[12] W. Huang, P. Abbeel, D. Pathak, 和 I. Mordatch。语言模型作为零-shot 规划器: 为具身智能提取可操作知识, 2022。[13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar 等。内心独白: 通过规划实现具身推理的语言模型。 *arXiv 预印本 arXiv:2207.05608*, 2022。

[14] J. Jung, L. Qin, S. Welleck, F. Brahman, C. Bhagavatula, R. L. Bras, and Y. Choi. Maieutic prompting: Logically consistent reasoning with recursive explanations. *arXiv preprint arXiv:2205.11822*, 2022.

[15] D. Kahneman. *Thinking, fast and slow*. Macmillan, 2011.

[16] D. Kahneman, S. Frederick, et al. Representativeness revisited: Attribute substitution in intuitive judgment. *Heuristics and biases: The psychology of intuitive judgment*, 49(49-81):74, 2002.

[17] G. Kim, P. Baldi, and S. McAleer. Language models can solve computer tasks, 2023.

[18] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+p: Empowering large language models with optimal planning proficiency, 2023.

[19] X. Lu, S. Welleck, P. West, L. Jiang, J. Kasai, D. Khashabi, R. L. Bras, L. Qin, Y. Yu, R. Zellers, N. A. Smith, and Y. Choi. Neurologic a*esque decoding: Constrained text generation with lookahead heuristics. In *North American Chapter of the Association for Computational Linguistics*, 2021.

[20] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[21] A. Newell, J. C. Shaw, and H. A. Simon. Report on a general problem solving program. In *IFIP congress*, volume 256, page 64. Pittsburgh, PA, 1959.

[22] A. Newell, H. A. Simon, et al. *Human problem solving*. Prentice-Hall, 1972.

[23] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[24] D. Paul, M. Ismayilzada, M. Peyrard, B. Borges, A. Bosselut, R. West, and B. Faltings. Refiner: Reasoning feedback on intermediate representations, 2023.

[25] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al. Improving language understanding by generative pre-training. *OpenAI blog*, 2018.

[26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[27] I. Schlag, S. Sukhbaatar, A. Celikyilmaz, W. tau Yih, J. Weston, J. Schmidhuber, and X. Li. Large language model programs, 2023.

[28] N. Shinn, B. Labash, and A. Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection, 2023.

[29] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[30] S. A. Sloman. The empirical case for two systems of reasoning. *Psychological bulletin*, 119(1): 3, 1996.

[31] K. E. Stanovich. *Who is rational? Studies of individual differences in reasoning*. Psychology Press, 1999.

[32] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[33] S. Verma, J. Fu, S. Yang, and S. Levine. Chai: A chatbot ai for task-oriented dialogue with offline reinforcement learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4471–4491, 2022.

J. Jung, L. Qin, S. Welleck, F. Brahman, C. Bhagavatula, R. L. Bras 和 Y. Choi. 逻辑提示：通过递归解释实现逻辑一致的推理。*arXiv* 预印本 *arXiv:2205.11822*, 2022年。

D. Kahneman. 思考，快与慢。麦克米兰，2011年。 D. Kahneman, S. Frederick 等人。再访代表性：直觉判断中的属性替代。启发式与偏差：直觉判断的心理学，第49卷（49-81页）：74，2002年。 G. Kim, P. Baldi 和 S. McAleer. 语言模型可以解决计算机任务，2023年。 B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas 和 P. Stone. Llm+p：赋能大型语言模型的最优规划能力，2023年。 X. Lu, S. Welleck, P. West, L. Jiang, J. Kasai, D. Khashabi, R. L. Bras, L. Qin, Y. Yu, R. Zellers, N. A. Smith 和 Y. Choi. 神经学类解码：带有前瞻启发式的受限文本生成。在北美计算语言学协会，2021年。 A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh 和 P. Clark. 自我优化：带有自我反馈的迭代改进，2023年。 A. Newell, J. C. Shaw 和 H. A. Simon. 一般问题解决程序报告。在*IFIP*大会，第256卷，第64页。匹兹堡，宾夕法尼亚，1959年。 A. Newell, H. A. Simon 等人。人类问题解决。普伦蒂斯-霍尔，1972年。 OpenAI. GPT-4技术报告。*ArXiv*, abs/2303.08774, 2023年。

D. Paul, M. Ismayilzada, M. Peyrard, B. Borges, A. Bosselut, R. West 和 B. Faltings. Refiner：中间表示的推理反馈，2023年。 A. Radford, K. Narasimhan, T. Salimans, I. Sutskever 等人。通过生成式预训练提升语言理解能力。*OpenAI*博客，2018年。 A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever 等人。语言模型是无监督的多任务学习者。*OpenAI*博客，2019年，第1(8):9页。 I. Schlag, S. Sukhbaatar, A. Celikyilmaz, W. tau Yih, J. Weston, J. Schmidhuber 和 X. Li. 大型语言模型程序，2023年。 N. Shinn, B. Labash 和 A. Gopinath. Reflexion：具有动态记忆和自我反思的自主代理，2023年。 D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton 等人。无需人类知识即可掌握围棋。*nature*, 550(7676):354–359, 2017年。 S. A. Sloman. 两套推理系统的实证依据。心理学通报，119(1):3, 1996年。 K. E. Stanovich. 谁是理性人？推理中的个体差异研究。心理学出版社，1999年。 H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar 等人。Llama：开源高效的基础语言模型。*arXiv* 预印本 *arXiv:2302.13971*, 2023年。 S. Verma, J. Fu, S. Yang 和 S. Levine. Chai：用于任务导向对话的聊天机器人AI，结合离线强化学习。在2022年北美计算语言学协会人类语言技术会议论文集，第4471–4491页，2022年。

[34] E. Wallace, N. Tomlin, A. Xu, K. Yang, E. Pathak, M. Ginsberg, and D. Klein. Automated crossword solving. *arXiv preprint arXiv:2205.09665*, 2022.

[35] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models, 2023.

[36] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[37] Z. Wang, S. Cai, A. Liu, X. Ma, and Y. Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents, 2023.

[38] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

[39] Y. Xie, K. Kawaguchi, Y. Zhao, X. Zhao, M.-Y. Kan, J. He, and Q. Xie. Decomposition enhances reasoning via self-evaluation guided decoding, 2023.

[40] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, and D. Schuurmans. Foundation models for decision making: Problems, methods, and opportunities, 2023.

[41] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[42] S. Zhang, Z. Chen, Y. Shen, M. Ding, J. B. Tenenbaum, and C. Gan. Planning with large language models for code generation. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Lr8c00tYbfL>.

[43] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

[44] X. Zhu, J. Wang, L. Zhang, Y. Zhang, R. Gan, J. Zhang, and Y. Yang. Solving math word problem via cooperative reasoning induced language models. *arXiv preprint arXiv:2210.16257*, 2022.

E. Wallace, N. Tomlin, A. Xu, K. Yang, E. Pathak, M. Ginsberg, 和 D. Klein. 自动填字游戏解题。 *arXiv preprint arXiv:2205.09665*, 2022年。

[35] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, 和 E.-P. Lim. 计划与求解提示：通过大型语言模型提升零-shot链式思维推理，2023年。

[36] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, 和 D. Zhou. 自治性提升语言模型中的链式思维推理能力。 *arXivpreprint arXiv:2203.11171*, 2022年。

[37] Z. Wang, S. Cai, A. Liu, X. Ma, 和 Y. Liang. 描述、解释、规划与选择：通过大型语言模型实现交互式规划，推动开放世界多任务智能体，2023年。

[38] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, 和 D. Zhou. 链式思维提示激发大型语言模型中的推理能力。 *arXivpreprint arXiv:2201.11903*, 2022年。

[39] Y. Xie, K. Kawaguchi, Y. Zhao, X. Zhao, M.-Y. Kan, J. He, 和 Q. Xie. 通过自我评估引导解码增强推理能力，2023年。

[40] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, 和 D. Schuurmans. 决策基础模型：问题、方法与机遇，2023年。

[41] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, 和 Y. Cao. ReAct：在语言模型中协同推理与行动。 *arXivpreprint arXiv:2210.03629*, 2022年。

[42] S. Zhang, Z. Chen, Y. Shen, M. Ding, J. B. Tenenbaum, 和 C. Gan. 利用大型语言模型进行代码生成的规划。发表于 *The Eleventh International Conference on LearningRepresentations*, 2023年。网址 <https://openreview.net/forum?id=Lr8c00tYbfL>。

[43] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le 等。最少到最多提示（Least-to-most prompting）实现大型语言模型的复杂推理能力。 *arXiv preprint arXiv:2205.10625*, 2022年。

[44] X. Zhu, J. Wang, L. Zhang, Y. Zhang, R. Gan, J. Zhang, 和 Y. Yang. 通过合作推理引导的语言模型解决数学应用题。 *arXivpreprint arXiv:2210.16257*, 2022年。

A Code, Prompts, Trajectories

All code is available at <https://github.com/princeton-nlp/tree-of-thought-llm>.
All prompts are available at <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/src/tot/prompts>.
Trajectories are available at <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/logs>.

B Additional Experiment Results

Given the motivation of exploring and extending the capability frontier of language models, our experiments in the main paper have focused on a setup with the state-of-the-art language model (GPT-4), and three hard tasks invented to challenge it. Here, we report additional experiments with weaker LLM or easier tasks, and discuss cost and efficiency.

GSM8K			StrategyQA			GPT-4			GPT-3.5		
IO	51	73	IO	7.3%	6%	IO	6.19	4.47	IO	6.19	4.47
CoT	86	82	CoT	4.0%	3%	CoT	6.93	5.16	CoT	6.93	5.16
ToT	90	83	ToT	74%	19%	ToT	7.56	6.62	ToT	7.56	6.62

Table 4: New tasks with zero-shot ToT and GPT-4.

Table 5: Game of 24 with GPT-4 vs GPT-3.5.

Table 6: Creative Writing with GPT-4 vs. GPT-3.5.

B.1 Extension to new tasks (GSM8k, StrategyQA) with zero-shot ToT

While more common NLP tasks might be too easy for GPT-4 and do not require ToT (which is why we considered harder new tasks), we believe applying ToT to new tasks could be straightforward. For example, we implemented a simple and generic zero-shot ToT-BFS similar to creative writing (sample 5 problem solving strategies then vote for the best one; then sample 5 solutions based on the best strategy then vote for the best one) for GSM8K and StrategyQA with few extra lines of code:

```
# define the answer format of new tasks
gsm8k_format = "the answer is n" where n is a number'
strategyqa_format = 'either "the answer is yes" or "the answer is no"'

# define zero-shot io prompting
standard_prompt = 'Answer the following question with {format}: {input}'

# define thought format for zero-shot cot and zero-shot tot
cot_prompt = 'Answer the following question: {input}'

Make a strategy then write. Your output should be of the following format:

Strategy:
Your strategy about how to answer the question.

Answer:
Your answer to the question. It should end with {format}.
'''

# define zero-shot voting used for zero-shot tot
vote_prompt = 'Given an instruction and several choices,
decide which choice is most promising.
Analyze each choice in detail, then conclude in the last line
"The best choice is {s}", where s the integer id of the choice.
'''
```

A 代码、提示、轨迹

所有代码均可在 <https://github.com/princeton-nlp/tree-of-thought-llm> 获取 。
所有提示均可在 <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/src/tot/prompts> 获取。
轨迹可在 <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/logs> 获取。

B 额外实验结果

鉴于探索和拓展语言模型能力前沿的动机，我们在正文中的实验主要集中在采用最先进的语言模型（GPT-4）以及三个设计用以挑战其能力的难题。在此，我们报告使用较弱的大型语言模型（LLM）或更易任务的补充实验，并讨论成本与效率问题。

GSM8K			StrategyQA			Tree of Thought: Different Problem Solving with Large Language Models 不同思维链的 问题解决策略与大型语言模型			Tree of Thought: Different Problem Solving with Large Language Models 不同思维链的 问题解决策略与大型语言模型		
IO	51	73	IO	7.3%	6%	IO	6.19	4.47	IO	6.19	4.47
CoT	86	82	CoT	4.0%	3%	CoT	6.93	5.16	CoT	6.93	5.16
ToT	90	83	ToT	74%	19%	ToT	7.56	6.62	ToT	7.56	6.62

表4：具有新任务的零样本 ToT 与 GPT-4。

表 5：24点游戏与 GPT-4 与 GPT-3.5。

表6：使用GPT-4与GPT-3.5进行创造性写作的比较

B.1 无需样本的零-shot ToT扩展至新任务（GSM8k, StrategyQA）

虽然更常见的自然语言处理任务对于GPT-4来说可能过于简单，不需要采用ToT（这也是我们考虑更难新任务的原因），但我们相信将ToT应用于新任务可能是直接的。例如，我们在GSM8K和StrategyQA上实现了一个简单且通用的零-shot ToT-BFS，类似于creativewriting（采样5个问题决策策略然后投票选出最佳策略；然后基于最佳策略采样5个解决方案并投票选出最佳方案），只需添加几行代码：

```
# 定义新任务的答案格式 gsm8k_format = "答案是 n"，其中 n 是一个数字'
strategyqa_format = '要么 "答案是是"，要么 "答案是否"'

# 定义零样本输入输出提示
s标准_提示 = '用{format}回答以下问题: {input}'

# 定义零样本链式推理（Zero-Shot Chain of Thought）和零样本思考（Zero-Shot Tree of Thought）思维格式_提示 = ' ' '回答下列问题: {input}'

制定策略然后写作。您的输出应遵循以下格式：

策略：
你关于如何回答问题的策略。

答案：
你对问题的回答。应以{format}结尾。

# define zero-shot voting 用于 zero-shot tot 投票_prompt = ' ' '给
定一个指令和多个选项，判断哪个选项最有潜力。对每个选项进行详细分
析，然后在最后一行得出结论：“最佳选择是 {s}”，其中 s 为选项的整数
编号。' ' '
```

We evaluated on a subset of 100 random GSM8K test and StrategyQA dev questions. As shown in Table 4 and as expected, ToT improves over CoT on both tasks (but only slightly, given GPT-4 + CoT is already very good on such tasks, and StrategyQA’s bottleneck is external knowledge, not reasoning). Considering computational costs, it is more suitable to try smaller LLMs + ToT for traditional NLP tasks, or GPT-4 + ToT for hard tasks that challenge GPT-4 + CoT’s reasoning.

B.2 Extension to new LMs (GPT-3.5)

To understand how ToT works with other LLMs, we also ran GPT-3.5-turbo for Creative Writing (Table 6) and Game of 24 (Table 5). On both tasks, “ToT > CoT > IO” remains true for GPT-3.5. On Creative Writing, we find GPT-3.5+ToT outperform GPT-4+IO, and similar to GPT-4+CoT, which suggests ToT could also work well on weaker language models.

On Game of 24 (we changed 1-shot proposal prompt to 3-shot to make it work), GPT-3.5+ToT’s 19% is far worse than GPT-4+ToT’s 74%. To further understand the importance of generation vs. evaluation, we ran GPT-4 generation + GPT-3.5 evaluation (64%) and GPT-3.5 generation + GPT-4 evaluation (31%). This suggests the game’s bottleneck is thought generation, and different generation/evaluation language models might attain decent results while reducing costs.

B.3 Cost and efficiency

Running ToT requires significantly more computations than IO or CoT prompting. For example, in Game of 24 (Table 7 below), solving a problem with ToT requires 5.5k completion tokens, close to 100 CoT trials (6.7k tokens). But the performance of ToT is better than best of 100 independent CoT trials.

Game of 24	Generate/Prompt tokens	Cost per case	Success
IO (best of 100)	1.8k / 1.0k	\$0.13	33%
CoT (best of 100)	6.7k / 2.2k	\$0.47	49%
ToT	5.5k / 1.4k	\$0.74	74%

Table 7: Cost analysis on Game of 24.

On Creative Writing (Table 8 below), we found ToT takes around 5x completion tokens and money cost, which is intuitive as $b = 5$ and most tokens are generated passages.

Creative Writing	Generate/Prompt tokens	Cost per case
IO	0.9k / 0.4k	\$0.06
CoT	0.9k / 0.4k	\$0.07
ToT	4k / 2.9k	\$0.32

Table 8: Cost analysis on Game of 24.

So completing Game of 24 and Creative Writing’s main ToT experiments cost around $0.74 \times 100 + 0.32 \times 100 = 106$ dollars. Crosswords’ DFS experiments should be also within 100 dollars. In general, cost and efficiency of ToT highly depend on the prompts and search algorithms used, and could require 5-100 times more generated tokens than CoT. Some actionable insights:

- We recommend using ToT on tasks requiring deliberate reasoning, on which CoT struggles.
- Flexibility of ToT allows some performance-cost tradeoff, e.g., change beam size or vote number in BFS, few-shot vs. zero-shot prompting, GPT-3.5 vs. GPT-4, etc. One could configure the setup based on some resource constraints or performance goal.
- There is much space for improving efficiency, e.g., BFS could early stop when solution is found, or trim down beam size to when some thoughts are ”impossible”.
- We believe that more computation is indeed required in order for the model to achieve stronger intelligence, and this should not become a blocking issue as in the long run, (open-source) LMs will become much cheaper and more efficient. It is also a great direction how to better train/finetune LMs for thought generation and/or evaluation.

我们在100个随机选择的GSM8K测试题和StrategyQA开发题的子集上进行了评估。如表4所示，正如预期的那样，Tree of Thoughts（ToT）在这两项任务上都优于Chain of Thought（CoT）（但提升幅度较小，因为GPT-4+ CoT在此类任务上的表现已经非常出色，而StrategyQA的瓶颈在于外部知识，而非推理能力）。考虑到计算成本，更适合尝试较小的大型语言模型 + Tree of Thoughts（ToT）用于传统自然语言处理任务，或GPT-4 +Tree of Thoughts（ToT）用于挑战GPT-4 + Chain of Thought（CoT）推理能力的难题。

B.2 扩展到新型语言模型（GPT-3.5）

为了了解ToT如何与其他大型语言模型（LLMs）协作，我们还对GPT-3.5-turbo进行了创意写作（表6）和24点游戏（表5）的测试。在这两个任务中，“ToT > CoT >IO”对于GPT-3.5依然成立。在创意写作任务中，我们发现GPT-3.5+ToT优于GPT-4+IO，并且与GPT-4+CoT表现相似，这表明ToT也可能在较弱的语言模型上表现良好。

在24点游戏（我们将单次提议提示改为三次提议以使其正常工作）中，GPT-3.5+ToT的19%远远低于GPT-4+ToT的74%。为了进一步理解生成与评估的重要性，我们进行了GPT-4生成 + GPT-3.5评估（64%）和GPT-3.5生成 +GPT-4评估（31%）的实验。这表明该游戏的瓶颈在于思维生成，不同的生成/评估语言模型可能实现不错的结果，同时降低成本。

B.3 成本与效率

运行ToT所需的计算量明显高于IO或CoT提示。例如，在“24点游戏”（下表7）中，使用ToT解决一个问题需要5.5k完成标记，接近100次CoT试验（6.7k标记）。但ToT的性能优于100次独立CoT试验的最佳结果。

24点游戏	生成/提示标记	每个案例的成本	成功
IO (100中最佳)	1.8k / 1.0k	\$0.13	33%
Tree of Thoughts Outperforms Problems Solving with Large Language Models: 6.7k / 2.2k CoT (100中最佳)	6.7k / 2.2k	\$0.47	49%
ToT	5.5k / 1.4k	\$0.74	74%

表7：24点游戏的成本分析

On 创造性写作（见下表8），我们发现ToT的完成标记大约是mone的5倍，这具有直观性，因为 $b = 5$ 且大部分标记是生成的段落。

创意写作	生成/提示标记	每个案例的成本
IO	0.9k / 0.4k	\$0.06
CoT	0.9k / 0.4k	B.3 成本与效率
ToT	4k / 2.9k	\$0.32

表8：24点游戏的成本分析

因此，完成“24点游戏”和“创意写作”主要ToT实验的成本约为 $0.74 \times 100 +0.32 \times 100 = 106$ 美元。填字游戏的DFS实验也应在100美元以内。总体而言，ToT的成本和效率高度依赖于所使用的提示和搜索算法，可能需要比CoT多出5到100倍的生成标记。一些可行的见解：

我们建议在需要深思熟虑推理的任务中使用Tree of Thoughts（ToT），而在此类任务中，链式推理（CoT）表现不佳。

- Tree of Thoughts（ToT）的灵活性允许在性能与成本之间进行一定的权衡，例如调整束搜索的束宽或投票数、少样本提示与零样本提示、GPT-3.5与GPT-4等。可以根据资源限制或性能目标对设置进行配置。在提高效率方面仍有很大空间，例如，BFS可以在找到解时提前停止，或在某些思路“不可行”时缩小束搜索的规模。我们相信，为了使模型实现更强的智能，确实需要更多的计算资源，而这在长远来看不应成为阻碍，因为（开源）语言模型将变得更加廉价和高效。这也是一个极具前景的方向，有助于更好地训练/微调语言模型以进行思维生成和/或评估。